



Scrambled text: fine-tuning language models for OCR error correction using synthetic data

Jonathan Bourne¹

Received: 8 October 2024 / Revised: 9 December 2024 / Accepted: 11 March 2025 / Published online: 17 April 2025
© The Author(s) 2025

Abstract

OCR errors are common in digitised historical archives significantly affecting their usability and value. Generative Language Models (LMs) have shown potential for correcting these errors using the context provided by the corrupted text and the broader socio-cultural context, a process called Context Leveraging OCR Correction (CLOCR-C). However, getting sufficient training data for fine-tuning such models can prove challenging. This paper shows that fine-tuning a language model on synthetic data using an LM and using a character level Markov corruption process can significantly improve the ability to correct OCR errors. Models trained on synthetic data reduce the character error rate by 55% and word error rate by 32% over the base LM and outperform models trained on real data. Key findings include training on under-corrupted data is better than over-corrupted data; non-uniform character level corruption is better than uniform corruption; more tokens-per-observation outperforms more observations for a fixed token budget. The outputs for this paper are a set of 8 heuristics for training effective CLOCR-C models, a dataset of 11,000 synthetic 19th century newspaper articles and `scrambledtext` a python library for creating synthetic corrupted data.

Keywords OCR · NLP · Post-OCR correction · Digital archives

1 Introduction

Optical Character Recognition (OCR) is a valuable tool in the historical archive digitisation process. OCR allows the text of archived material to be digitised at character level, allowing archival documents to be searched and manipulated like more modern natively digital documents. However, performing OCR on archival documents is not without errors [1]. These errors can be particularly acute in archival media such as newspapers and periodicals due to their complex layouts [2]. Such errors impact the value of the digitisation process as the ability to search for individuals or specific places and events can degrade dramatically [2–4].

As a result of the issues with the OCR processes, post-OCR correction has become an area of long time active research [5–7]. More recently, with advances in machine learning and the development of the transformer architecture [8], the use of transformer-based language models

for post-OCR correction has grown in popularity [9–11]. Transformers offer additional benefits over more traditional approaches to OCR correction [6, 7], in that a prompt can be provided with useful context such as socio-cultural information (‘This is an 19th century newspaper’, ‘This is a love poem’), as well as being able to use the Transformers facility to handle long-range textual dependencies to provide context on corrupted characters and words. Such an approach to OCR correction can be called Context Leveraging OCR Correction (CLOCR-C). Pre-trained language models have been shown to reduce errors by over 60% [11]. However, the SOTA (State-Of-The-Art) models are the large frontier models such as Claude or GPT, which can be prohibitively costly for large amounts of data. An alternative approach is to fine-tune smaller open-source models [9, 12] specifically on the task of CLOCR-C.

Fine-tuning is a process of performing additional training on a pre-trained LM to make it better at a specific task (See [13] for examples of the effectiveness of fine-tuning). However, the process can be expensive and time-consuming. Recent research suggests that much of the functionality of transformer-based LMs exists on a relatively small sub-space of the LM parameters and that by fine-tuning a small subset

✉ Jonathan Bourne
jonathan.bourne.15@ucl.ac.uk

¹ Centre for Advanced Spatial Analysis, University College London, Gower St, London WC1E 6BT, UK

of parameters on a specific task, the model can experience significant improvements in that task using much less compute. These processes are collectively known as Parameter Efficient Fine Tuning (PEFT) [14, 15]. One of the most popular approaches to PEFT is Low-Rank Adaptors (LoRA) [13], which, when combined with other recent advances, such as floating point quantization [16], substantially reduces the cost of fine-tuning, due to reduced GPU use, whilst also providing regularization to prevent overfitting [17]. For example, Quantized LoRa Llama2 7B required RAM drops from 38.7Gb to 7.5Gb, and token throughput increases 20% relative to the base Llama2 model [18].

The fine-tuning process is dependent on having data in sufficient quantity and quality to be able to improve the performance of the model, whilst there are several appropriate datasets available [5, 11, 19–22], this does not mean they have the necessary qualities to act as training material. Recent research has highlighted that noisy or poor-quality data can impact the performance of trained LMs [23–27]. Such noise is a problem when it comes to creating ground truth datasets for OCR correction (as noted by [19]). One way to avoid this problem is using expert transcribed datasets [11, 19, 22]. However, transcription is time-consuming and/or costly, and using previously transcribed data may not have the correct error distribution or sufficient corruption [28] to be of value.

An alternative to transcribed data is to use synthetic data created using a noising or corruption process [27–29], as previous work has shown that OCR errors have identifiable patterns [30]. Recent advances in training LMs suggests that synthetic data can play a crucial role in training and fine-tuning LM models due to the guaranteed high-quality nature of the data [31–33]. However, synthetic data needs to be applied carefully, as there is evidence that when LMs are trained on recursively generated synthetic data, they can suffer model collapse and produce gibberish [34]. Whilst synthetic data has been shown to be valuable in various LM applications, little is known about how such an approach would affect the final performance of LMs in CLOCR-C.

This paper contributes to the literature by demonstrating that synthetically generated text, corrupted using a learned function, can improve the performance of LMs to correct OCR errors. Specifically, it answers the following questions:

- Can synthetic corrupted OCR data be used to train a language model such that it is better able to perform CLOCR-C?
- How does the level of corruption of the synthetic data impact the performance of the fine-tuned language model?
- What are a set of basic guidelines for practitioners using synthetic data for fine-tuning a CLOCR-C LM?

The rest of the paper is as follows, Sect. 2 introduces the dataset used in this paper, Sect. 3 describes the methods for creating the synthetic text and the experiments used to measure its effectiveness as training data, Sect. 4 reports the results of the experiments, Sect. 5 is the discussion interpreting the findings and their place relative to previous literature, Sect. 6 provides a list of heuristics for training CLOCR-C models, and finally Sect. 7 provides the conclusions and considers future work.

2 Data

When fine-tuning LMs for OCR correction, data selection must be done carefully. This is because much open-source data has already been used to train the most high-performing LMs, which can lead to the memorisation of the text and apparently exceptional recovery rates due to data leakage. As a result, much potential training data is ruled out, such as [21] as it is based on data from the Gutenberg project [35], which is used in training many LMs. As such, this paper will use 4 OCR correction datasets transcribed from archive newspapers, which are unlikely to have been used for training because they have been, specifically transcribed and not yet released publicly [11], are public but highly inaccessible [36], or are stored specifically to discourage being picked up by web crawlers [22]. Using archival newspaper data is also interesting due to the high levels of corruption that they can tend to exhibit and the fact that the data will all be from a similar distribution regarding style and structure. The datasets used are described below.

- NCSE: This is a 40,000-word corpus [11] made up of 91 articles from the original Nineteenth-Century Serials Edition.
- SMH: These are articles from the Sydney Morning Herald containing 52,000 words and 159 articles created by [36] using the TROVE dataset [37].
- CA: Chronicling America, an 18,000-word dataset across 46 articles by [36]
- BLN600: A 294,000 word collection of 600 articles from 19th century English newspapers from [22], this paper will also use the sequence level dataset created by [9].

Of these 4 datasets, the NCSE dataset will be used as the test dataset as it has the highest overall error rate and is an excerpt from a much larger dataset that would benefit from CLOCR-C. The other three datasets will be used to train the corruption function.

3 Method

This section is broken into 5 parts. The first part describes the evaluation and metrics used in the paper followed the Markov corruption process. After the Markov process is introduced the synthetic data generation process is defined. Then, the Llama LM architecture and training parameters are introduced and explained. Finally, the experiments are described.

For clarity, Fig. 1 shows the flow diagram of the method. The diagram has three types: Data (blue), such as the line-aligned texts; Processes (pink), such as corrupting the text; and Computational tools (green), such as `scrambledtext`. Each element of Fig. 1 will be described in detail.

3.1 Evaluation

This paper evaluates the performance of the LMs using the Character Error Rate (CER) and the Word Error Rate, which are calculated using

$$ER = \frac{S + D + I}{S + D + C} \quad (1)$$

Where S is the number of substitutions, D is the number of deletions, I is the number of insertions, and C is the correct total number, the denominator is equivalent to the number of characters in the ground truth reference document. The modifications refer to characters or words depending on whether the metric is CER or WER. In this paper, the reported error rate for a dataset is the error of the median article. The original CER for each dataset used in the paper is NCSE 0.17, SMH 0.08, CA 0.1, and BLN600 0.07. The WER is only used to measure the improvements to the NCSE, whose original WER is 0.62.

Having defined the metrics used to measure corruption, we will next introduce the corruption function itself.

3.2 ScrambledText: creating the Markov corruption process

The corruption generation function aims to simulate the character corruption type found in OCR documents. Previous research has produced high-quality corruption processes [27, 28]; however, these focus on typographical or human errors, not OCR-style errors. Other similar approaches, such as back-translation [38], cannot control the amount of error. Only Genelog [39] is explicitly designed to re-create OCR errors, which it does by creating a synthetically corrupted PDF and then performing OCR on it. Whilst very thorough, this approach is computationally expensive and technically tricky due to automatically generating appropriate layouts, making it impractical for low-resource projects.

This subsection is divided into three subsubsections, the first describes and defines the Markov Model, the second discusses the implementation of the model in `scrambledtext`, and the third discusses the application of the model in this project and provides example text.

3.2.1 The Markov model

Due to the issues with the existing methods, this paper develops `scrambledtext`, a simple, fast corruption process based on a Markov model specifically designed to simulate OCR corruption errors using a straightforward training process. The model can output text with an arbitrary amount of corruption. The Markov Model is shown in Fig. 2, and the corresponding algorithm is shown in Algorithm 1. The model takes a single correct character x and outputs a sequence of characters y with a length greater than or equal to 0. The model begins by choosing the corruption state and using the conditional probability distribution for character x , which has been learned from the training data. If the initial corruption state returned is 'Correct' or 'Delete', the process terminates, returning the character x or no character, respectively. In the case the initial corruption state returned is 'Substitute', then the conditional probability table for the substitution distribution of character x is sampled to replace x with x' ; once the substitution has taken place, the corruption state table is resampled conditioned on x' . If the returned state is 'Insert', the insertion process begins based on the substituted character x' . Otherwise, the process terminates by returning x' . If the initial or the post-substitution corruption state returned is 'Insert', then the conditional probability insert table for character x is sampled and x' inserted after x . Similar to the substitution case, the conditional corruption state table is then randomly sampled again. If the state returned is 'Insert', the process repeats or terminates otherwise. It should be noted that because a Markov network is memoryless, the output x' of the substitute or insert node is treated identically to x by the insert node.

Once the process has terminated, the next character in the character string is processed using the same method. Because each character is independent, the probability of the resulting text string can be represented as

$$P(Y|X) = \prod_{j=1}^{j=r} P(y_j|x_j) \quad (2)$$

Where X is the input text r characters long, and Y is the output text resulting from each character in text X passing through the corruption model.

The Markov structure shown in Fig. 2 and Algorithm 2 allows the process to be broken into two stages and three conditional probability tables. The stages are state identifica-

Fig. 1 The relationships between the elements of the project are visualised as a flow diagram. In this diagram the data is shown in blue, the processes in pink and the computational tools such as scrambledtext and Llama as green (color figure online)

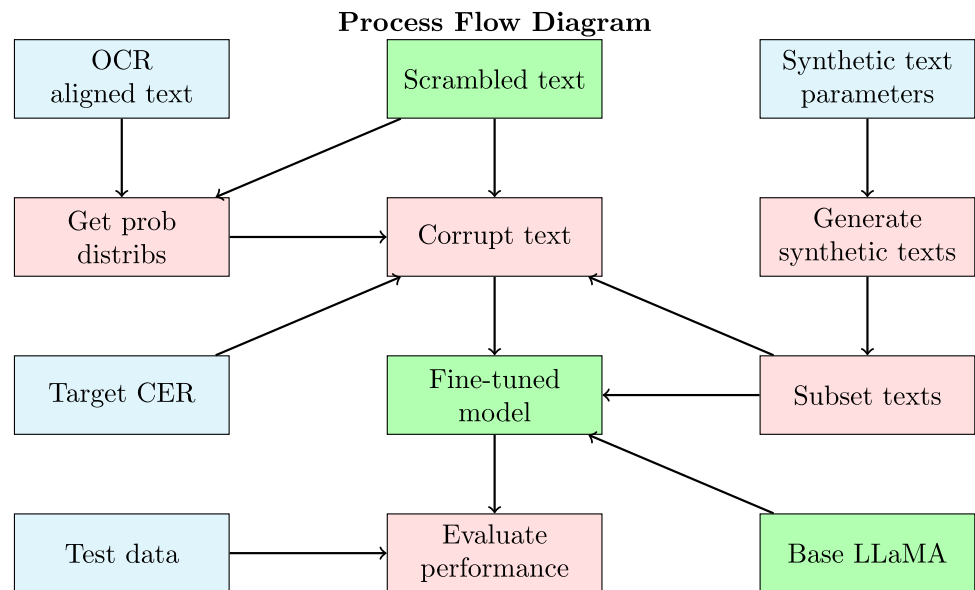
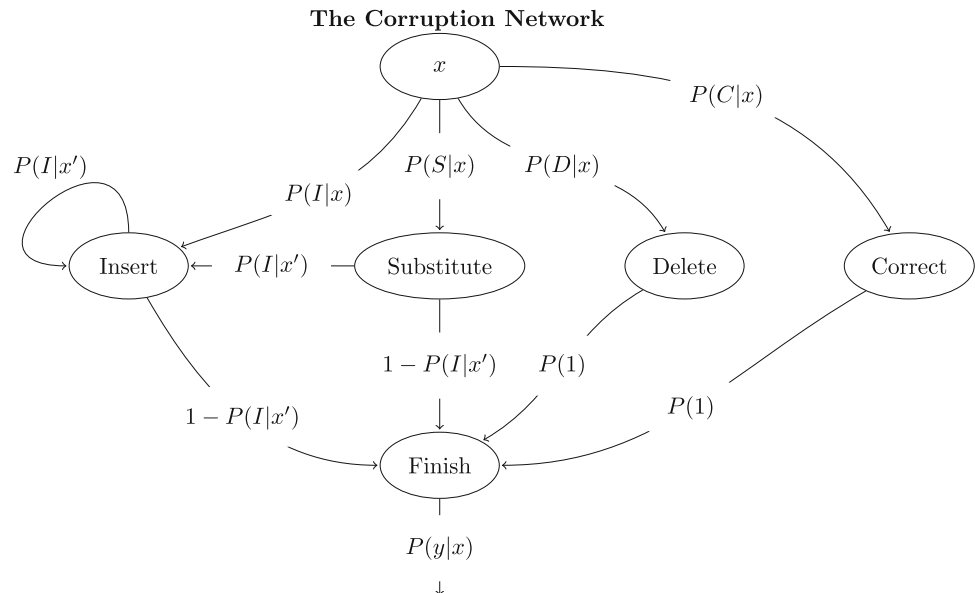


Fig. 2 The corruption network is applied at character level to the text; the conditional transition probabilities between the nodes are learnt on a per character basis from aligned OCR and ground truth texts



tion and corruption. The probability tables are the conditional state probability table, the conditional substitution table, and the conditional insertion table. These conditional probability tables can be calculated by learning the transmission probabilities from OCR text and corresponding ground truth. The algorithm for this learning process is shown in Algorithm 2. The basic learning process of the algorithm is that it simply counts occurrences of the error type, substitution character, and insertion character for all ground truth characters, then normalises using the counts of the ground truth character. After training, for each of the state, insertion, and substitution tables, the mean distribution, marginalised across all

ground truth characters, is found and added to its respective table to allow handling of characters unseen during training.

A significant advantage of the proposed Markov corruption model is that the Target CER can be easily chosen as $CER \approx 1 - P(C)$, where $P(C)$ is the marginal probability of being correct. Consider the conditional state matrix M with n rows representing the ground truth characters and four columns representing the four possible character states $j \in \{C, D, S, I\}$. For the i th character in the ground truth where $1 \leq i \leq n$ the conditional state probabilities are $P(C|i)$, $P(D|i)$, $P(S|i)$, $P(I|i)$. When the target CER is chosen, $P'(C) = P'(C|i) = 1 - (\text{Target CER})$, where $P'(C)$ is the modified probability of a character being

Algorithm 1: Algorithm of the Markov corruption network

Input: character x
Result: List of characters
 $y \leftarrow [x];$
 $i \leftarrow 0;$ // number of errors
 $state \leftarrow \text{"correct"};$
while true do
 // An infinite loop until 'return'
 condition met
 if $state = \text{"correct"}$ **then**
 $state \leftarrow \text{sample_state}(y[-1]);$ // $y[-1]$ the last
 element of the list
 if $state = \text{"correct"}$ **then**
 return $y;$
 end
 else if $state = \text{"delete"}$ **then**
 return $[];$
 else if $state = \text{"substitute"}$ **then**
 $y[i] \leftarrow \text{sample_substitute}(y[i]);$ // conditioned
 on $y[i]$
 $i \leftarrow i + 1;$
 $state \leftarrow \text{sample_state}(y[-1]);$
 if $state \neq \text{"insert"}$ **then**
 return $y;$
 end
 else if $state = \text{"insert"}$ **then**
 $i \leftarrow i + 1;$
 $y[i] \leftarrow \text{sample_substitute}(y[i-1]);$
 $state \leftarrow \text{sample_state}(y[-1]);$
 if $state \neq \text{"insert"}$ **then**
 return $y;$
 end
 end
end

correct marginalised across all n characters, and $P'(C|i)$ is the modified probability of the i th character being correct. The elements of the conditional state matrix M can then be rescaled to create M' using

$$M'_{ij} = \begin{cases} M_{ij} \cdot f_i & \text{if } j \neq C \\ P'(C) & \text{if } j = C \end{cases} \quad (3)$$

where f_i is the scaling factor for the i th character and is found by

$$f_i = \frac{1 - P'(C)}{1 - P(C|i)} \quad (4)$$

for each of the n characters in M . The scaling factor ensures that the probability mass of each state is proportionally scaled. As a result of this re-scaling, the state conditional probability matrix M' has a constant $P'(C) = P'(C|i)$ for all n characters. Other states are scaled $P'(D|i)$, $P'(S|i)$, and $P'(I|i)$ but still related to the original training distribution.

Algorithm 2: Train corruption model conditional probabilities

Input: paired_texts: List of (OCR_text, GT_text) pairs
Output: state_matrix, substitution_matrix, insertion_matrix
Initialize empty $M_{state}, M_{sub}, M_{ins};$
foreach $(ocr_text, gt_text) \in \text{paired_texts}$ **do**
 $aligned_text \leftarrow$
 perform_line_alignment(ocr_text, gt_text);
 foreach character $x \in aligned_text$ **do**
 Update $M_{state}[x]$ with observed state;
 if x' substitutes x **then**
 $M_{sub}[x \rightarrow x'] += 1;$
 if x' is inserted after x **then**
 $M_{ins}[x \rightarrow x'] += 1;$
 foreach $M \in \{M_{state}, M_{sub}, M_{ins}\}$ **do**
 Normalize rows to create conditional probability distributions
 $P(x'|x);$
 Add unknown character distribution as mean of all rows;
return $M_{state}, M_{sub}, M_{ins};$

In the edge case that $P(C|i) = 1$, the conditional probability distribution is left unchanged.

This approach can be critiqued as being a somewhat blunt instrument in that the corruption function does not consider more than a single character at a time, thus missing common errors such as $m \rightarrow rn$. Still, it is conceptually easy to understand and computationally cheap, making it ideal for exploring synthetic data for training LMs to perform CLOC-C.

An in-depth exploration of the implications of the corruption model is beyond the scope of this paper. However, it is worth making two points. The first point is that because the corruption function may be applied to a text with a different character distribution than that with which it has been trained, the target CER and the observed CER after corruption may differ. The second point is that, as the output sequences are a probability distribution, the observed CER for a uniformly corrupted text will itself have a probability distribution centred approximately on

$$P(\text{CER}|X) \approx 1 - \sum P(C|x)P(\hat{x}) \quad (5)$$

which is one minus the joint probability of the learned conditional probability table $P(C|x)$, and the observed character distribution $\hat{x}$ from the text X .

3.2.2 Process implementation

The corruption function has been implemented in Python as a library called `scrambledtext`. The process requires only parallel OCR texts that have been line-aligned; the learned conditional corruption probabilities can be easily

Table 1 A well know phrase by Ada Lovelace [40], is used to demonstrate the effect of corrupting with scrambled text. As can be seen as the corruption function increases the CER, the text becomes increasingly illegible

Target CER	Observed CER	Corrupted text
0.00	0.00	We may say most aptly that the Analytical Engine weaves algebraical patterns just as the Jacquard-loom weaves flowers and leaves
0.10	0.09	We may say mot aptly tAt tho Analytical Engine wedaves Talgebraical patterns just as theJacquard-lom weaves fowrs anld leavesi
0.20	0.20	We may san mest puly thab th e Analytical Engrte weFaves ealgebr'airal Fpttens just a ihe J acquard-loofwcves flowers andt lleaves. 1
0.30	0.28	Wcelay say mtost apteJy -thnt tha AndasyTtical Elninewcaves algeibi-acalf pattetnsa yus t -as t heo.Jaeqaard-loomweaovesflowers an-d leaves
0.40	0.47	ge mnay dsa most arpti tha-ihie A natlyei..cil Ea-gire Twiealyes rglbrd;ienl settias ojis as- 'lhe Jtcnad-loom wteaveml lloweia iand Ple-ave.'
0.50	0.62	W.e may 'sav tosmt aptdy rll ltiit fSe k.nal ricll-En-ggn taa-we,\eragoeibraac lplate,,lns g"a- aa.thie oJsciaa.rd.-loo-ml Cvv,evosfl-owerls aad ave

Table 2 The prompt is populated by replacing the bold words in curly braces with appropriate descriptors

Synthetic text generation prompt
It is the year {year}. Using the text provided below surrounded by triple #, write a {word_count} word {writing_style} {text_type} with a {sentiment} sentiment, the persona of the writer is {persona}, the reading level should be {complexity}.
Note: The resultant text may be distasteful to modern readers, that is ok. Respond only in plain text, do not use markdown
###
{text}
###

exported as JSON files and passed between projects. In addition, as the process is based on adjacent symbols, the process is language and script agnostic. The library has two main classes, *ProbabilityDistributions*, which learns the corruption conditional probability distributions, and *CorruptionEngine*, which uses the learned corruption distributions to corrupt text. The library is lightweight, relying only on Python libraries included in the standard distribution, and is easily extensible using class inheritance.

3.2.3 Application of the model

The model is trained using the BLN600, SMH and CA datasets, which are used to try to create diversity in the observed errors. Each dataset's OCR and ground truth texts are character aligned using the *genalog* [39]. The texts are then sequentially loaded, and the conditional distributions are learned. Once the distributions have been learned, they are saved as a JSON file for use in the rest of the project. With the conditional probabilities learned, text can now be corrupted to an arbitrary level. Table 1 shows an example of text [40] being corrupted from CER = 0 to CER = 0.5. Table 1 shows both the Target CER that the corruption function was aiming for and the Observed CER of the output sequence and drift from character insertions. The difference is due to

the stochastic nature of the process and the difference in the character distribution from the distribution of characters in the training text.

Having defined the corruption method, the process for creating the synthetic data will now be described.

3.3 Synthetic data

The data is created by prompting a Language model (in this case GPT-4o) to write a piece of text under a set of guidelines. The prompt is shown in Table 2.

The seven words inside curly braces, shown in Table 2, are the variables used to create diversity between the texts. The seed prompt, shown as '{text}' in the prompt, was obtained by downloading the Wikipedia timeline of the 19th century and the timeline of British diplomacy in the 19th century; combining both lists created 594 events to describe. These timelines provide a brief description of an actual 19th-century event, which could then be styled by the rest of the prompt. The variables used for prompt styling were chosen to be appropriate for the 19th century and are shown in Table 3; there are 3888 possible combinations of text styling. The timeline and prompt variables were then sampled 11,000 times, and the text generated by GPT-4o. This resulted in a corpus of 11,000 texts with 3.5 million words.

Table 3 Variables and options used in the prompt template

Variable	Options
Text type	Newspaper article, obituary of a named person, editorial, book excerpt, letter to the editor, personal diary entry
Writing style	Formal, informal, satirical, religious, polemic, romantic, persuasive, descriptive
Persona	General public, women's rights, politics, economics and trade, military, reactionary, chartist, clergy, arts and culture
Sentiment	Positive, neutral, negative
Complexity	Simple, moderate, advanced

Each synthetically generated text was sub-set to a 200 token long sub-string selected from a random point in the text, at least 200 tokens from the end. Each sub-string acts as a single observation in dataset. In this paper tokenization uses the Llama 3 tokenizer which is based on tiktoken.¹ Sub-setting to a portion of articles means that the model does not expect an entire article, speeds up training, and allows the model to fit onto a 24Gb GPU. The sub-setting process is used to create a 10,000-observation training set where each observation is of consistent length. The remaining 1000 observations are split between a test and validation set, but are not used in this project. An example of a prompt and the resultant article are shown in Supplementary material Section 3, the synthetic data can be downloaded from [41].

Having provided an overview of the dataset and evaluation metrics, the models and training parameters used throughout the experiments is introduced.

3.4 The Llama model and training parameters

This paper constrains the number of experiments by not attempting to optimise the model's performance but rather to explore the influence of the training with synthetic data on performance. This data-centric modelling approach [42–44] means that we need only evaluate one model architecture and that the parameters of the model can be fixed so that the impact of changes in the data is clear.

We use the Llama–3.1 8B instruct [45] an 8 billion parameter model which, at the time of writing, was one of the top performing models of its size class. Like most current top-performing models, Llama is an autoregressive (or causal) decoder-only model based on the transformer architecture. In addition, the model has an open licence, making fine-tuning and distribution of the resultant models possible.

In relation to the discussion of PEFT in Sect. 1, this paper will use LoRA [46] to reduce parameter count of the

Table 4 Selected hyper-parameters used in the model

Hyper-parameter	Value
LoRA Rank	64
LoRA Alpha	32
Learning rate	5e–5
Epochs	1
Scheduler	AdamW 8-bit
Batch size	16
Context window	1024

Llama mode and so reduce RAM requirements and training time. Specifically Rank Stabilised LoRA [47] which has been shown to produce better results than the original LoRA approach.

The models are trained using the Huggingface framework² and the Unsloth [48] implementation of Llama. Training is performed on the lightning.ai platform,³ the GPU will be a 24Gb Nvidia L4. Table 4 shows the most important hyper-parameters used in the training. The hyper-parameters were chosen to ensure that the model would not exceed the 24Gb RAM of the GPU; from this perspective, setting the context length was most important. From experiments a token length of 1024 was chosen as this would be able to handle all levels of corruption used in the paper, detail on this choice can be seen in supplementary material Figure 1. After an appropriate context window was found, the batch size and LoRA parameters could be chosen. A full list of hyper-parameters is available in the code (See Data Availability section at the end of the paper).

Although fine-tuning reduces the importance of the prompt it is still valuable, all models will be fine-tuned using the same prompt which will help frame the socio-cultural context of the task. The prompt is as follows

You are an expert in recovering OCR text, please recover the below text from an 19th century British Newspaper. End the recovery with triple ###

This prompt provides the socio-cultural context of the task, and also provides a terminating clause, this is useful as sometimes models can add additional discussion. Applying the triple hash reduces the risk of additional text getting mixed into the recovered result.

With the experimental framework defined, the experiments themselves are introduced and defined.

² www.huggingface.co.

³ www.lightning.ai.

¹ <https://github.com/openai/tiktoken/tree/main>.

3.5 Experimental setup

This subsection describes the 3 different experiments performed in this paper, producing a total of 68 models. First, the level of corruption and the distribution of corruption is explored. Then, the impact of the length of text in relation to the number of observations is investigated. Finally, a comparison between models trained on synthetic data and real data is performed.

3.5.1 Corruption level, and corruption distribution

When using synthetic data to fine-tune an LM to learn how to do CLOCRC, there are two key questions:

1. How much synthetic error is appropriate?
2. How should the error be distributed?

This paper explores the topic using three experiments: first, by simply uniformly increasing the CER across the dataset; second, by varying the relationship between CER and WER; and finally, by combining CER and WER levels within the dataset. A more detailed description of the parameters are shown in the bullet points below. Given the extreme levels of corruption shown for CER = 0.5 shown in Table 1, having a CER level much beyond 0.4 loses almost all the original text, becoming unrecoverable nonsense; as such, this paper will focus on the lower end of the CER scale. The question this series of experiments seeks to answer is “what corruption parameters provide the biggest improvement to model performance?”

1. Data is corrupted with a fixed conditional probability such that the mean CER of the text is one of nine possible values, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9.
2. Using a grid of WER-CER pairs, data is corrupted across a WER range of 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7 for a target CER of 0.05, 0.1, 0.2, 0.3, 0.4.
3. Instead of models being trained on data which has all been corrupted in the same way, the model is trained on a mixture of corruption types to increase error diversity. Each training observation is randomly assigned to one of four CER levels (0–0.1, 0.1–0.2, 0.2–0.3, and 0.3–0.4) such that each level makes up 25% of the total training set. The WER level will be controlled by sampling from the empirical WER distribution for that CER group from the combination of the BLN600, CA and SMH datasets (See supplementary materials Section 4). A second model will be created the same as described but with uncorrupted data as a fifth group, each group making up 20% of the dataset.

Table 5 The models will be trained using different combinations of text-length and token budget. The table shows the number of data observations for a given observation token length and total token combination

Total Tokens	Tokens per observation				
	200	100	50	25	10
1,638,400	8,192	16,384	32,768	65,536	163,840
819,200	4,096	8,192	16,384	32,768	81,920
409,600	2,048	4,096	8,192	16,384	40,960
204,800	1,024	2,048	4,096	8,192	20,480
102,400	512	1,024	2,048	4,096	10,240
51,200	256	512	1,024	2,048	5,120
25,600	128	256	512	1,024	2,560

This set of experiments will produce 46 models, 9 CER-only experiments, 35 CER-WER experiments, and 2 multi-CER-WER models.

It should be noted that as CER increases for a given WER, the relative corruption of the words being corrupted increases disproportionately. This means the document CER and the CER of the corrupted words may differ, which may play a role in the fine-tuned model’s overall performance. One can say that the CER of the corrupted words is the “effective CER” of the text. For the corruption function used in this paper, the effective CER saturates at 1.5. When CER is saturated, the solution to the problem is similar to masked language modelling, given the assumption that the text is all real words. However, a deeper exploration of these links is beyond the scope of this paper. More details on the relationship between CER-WER pairs and effective CER can be found in supplementary material in Figure 2.

3.5.2 Text-length vs number of observations

Given that a CER-WER combination can be found that provides sufficient improvement in the test set, it is valuable to investigate whether other factors beyond the corruption type play a role in the performance of the LM at CLOCRC. Two critical elements in the training of the LM are the length of the text supplied and the number of observations. This will be explored by modifying the total number of tokens in the training set and also the total number of tokens per observation. These two parameters will be varied according to Table 5. This experiment will produce 36 token text-length pairs, from which we will gain insight into the importance of text-length and token volume. An example of the different token lengths is shown in supplementary material section 3.

3.5.3 Between dataset comparison

Three models are created, one for each of the datasets, BLN600, SMH, and CA. The performance of these models

will be compared to synthetic data, the original dataset, the Base Llama3 model, and the current state-of-the-art Claude Opus. This comparison will provide insight into the relative performance of synthetic data vs available training data. The CA and SMH data have been transcribed with line breaks, making splitting the datasets easier; these two datasets will be split such that the median tokens per observation in the ground truth are 200, the same as the synthetic training set. As SMH and CA are small datasets, they will also be combined into a single Overproof dataset named after the project in which they were transcribed. The BLN600 data is not line aligned but has already been turned into a dataset using sequence matching [9], however, the median tokens per sequence in the ground truth dataset is very short being only 26 tokens.

4 Results

All models with uniform synthetic CER reduced the WER on the NCSE dataset. The CER was minimised when the synthetic training data had a CER of 0.2, close to the median CER of 0.17 of the NCSE dataset, as well as the 0.05 model. As a control the NCSE OCR text was spell checked using the python `spellchecker` library, and the CER and WER Measured. The spellchecked CER was 0.18, and the WER was 0.62.

Figure 3 shows the performance of the models trained on synthetic data from different CER-WER combinations the colour of the point indicates the level of synthetic corruption the model was trained on. The performance of the baseline Llama3 model is shown with solid red lines whilst the original NCSE dataset median is shown with a dashed red line. The horizontal distance between a vertical line and a point shows the improvement in CER post correction for that model, the vertical distance from a horizontal line to a point shows the improvement in WER. The figure shows that whilst all models reduced error compared to the baseline WER and most models beat the Baseline Llama3, reducing the CER relative to the NCSE dataset was much more difficult. Primarily, models trained with low levels of CER were more likely to reduce the overall CER; however, there was not such a clear pattern with the WER values. A key factor of OCR errors is that the range of errors can be large and have considerably different impact on the text (see Table 1 for examples). The median CER of the top ten models was 0.13 and the WER was 0.28; which is an error reduction percentage of 24% and 55% respectively relative to the NCSE test dataset, and an improvement of CER = 55% and WER = 41% relative to the Baseline Llama model. An example of a recovered article is given in supplementary material section 5.

The text was split at the median CER into high and low corruption subsets to explore the practical significance of

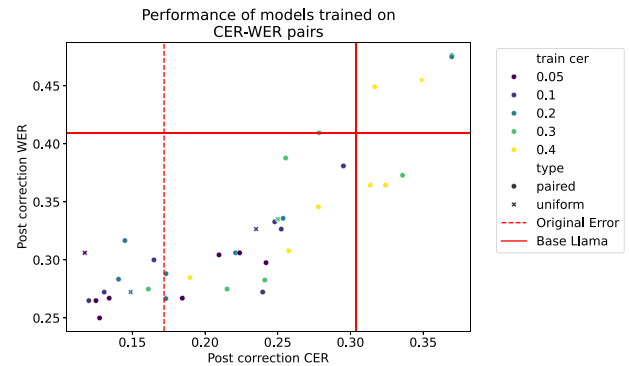


Fig. 3 Solid red lines show the performance of the baseline Llama3 model, and the dashed red line shows the median CER of the NCSE dataset. Across the whole NCSE dataset, most models outperformed the baseline Llama3. Still, few managed to reduce the CER compared to the NCSE median of 0.17 (color figure online)

the difference in overall CER on model performance. The high-low split was at CER = 0.17, shown in Fig. 4. What Fig. 4 reveals is that there is a very large difference in the corruption, with the lower group having a median CER of 0.06 compared to the high corruption group's CER of 0.61. Models that perform well in the low corruption group all have very low target CER values, typically 0.05 or 0.1.

In other words, the base Llama model increases the median error of the OCR data on average across the dataset except if the dataset is split into low and high corruption halves, in which case the base Llama model improves BOTH the high and low corruption sub-sets. This situation appears bizarre and reminiscent of Simpson's paradox [49, 50], a statistical phenomenon in which trends observed in sub-populations is reversed when the data is combined into the total population. However, it can be explained by considering the CER distribution of the NCSE dataset as a Gaussian-mixture model of two components. When partitioned at the midpoint, the lower distribution is heavily right-skewed, with the centre of mass close to the lower limit, whilst the upper distribution is more normally distributed, with the centre of mass shifted away from the 0.17 lower limit. When viewed as a whole dataset, the mean CER of the base Llama3 model and the original dataset are both approximately 0.32, but their median values diverge.

In addition to the paradoxical appearance of model performance, the types of fine-tuned models that perform well are also very different. The low-corruption dataset was improved most by models trained on low-corruption data, whilst the high-corruption dataset was improved most by models trained on moderately corrupted data. This is interesting as it adds nuance to the findings of [28], who suggested that corruption should be at least as bad; these findings indicate that there may be an upper limit to corruption for successfully training models to correct OCR.

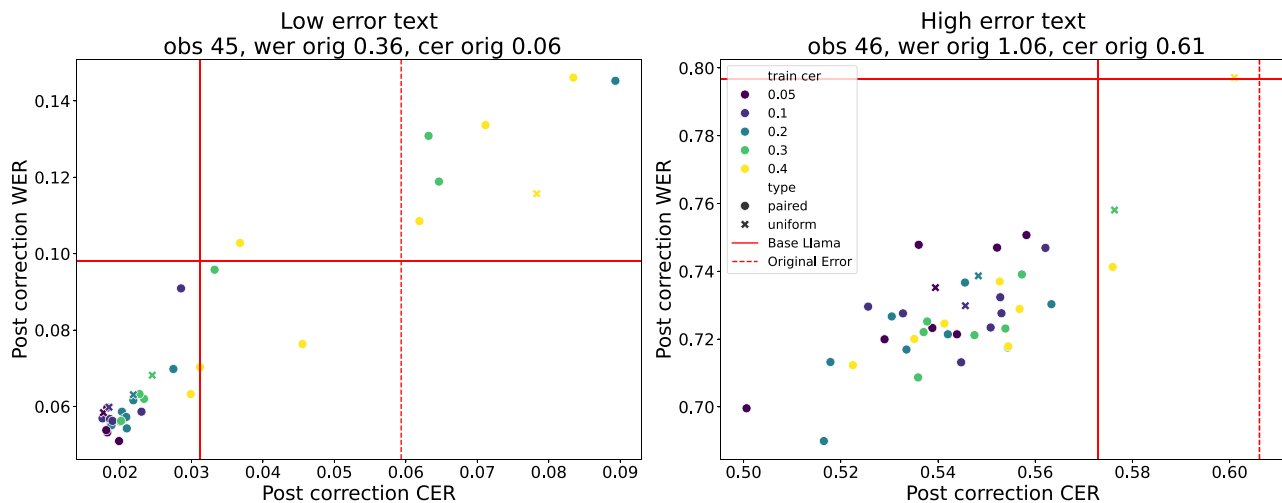


Fig. 4 The figure shows the contrast in performance when looking at only the high ($\text{CER} > 0.17$) and low ($\text{CER} \leq 0.17$) corruption. Solid red lines show the performance of the baseline Llama3 model, and the dashed red line shows the median CER of the NCSE dataset (color figure online)

4.1 Tokens per observation and total tokens in dataset

Given the results discussed in the previous section, the model trained with $\text{CER} = 0.1$ and $\text{WER} = 0.2$ was chosen as the base for the tokens-per-observations and tokens-in-training-set experiment. The results of the experiment can be seen in Fig. 5 which shows the interaction between the two variables. Looking at the left panel of Fig. 5 the relationship between number of tokens and CER is difficult to see, it appears that there is substantial noise in the results, with only 200 tokens per observation models showing any discernible trend. This contrasts with the right panel, which shows how CER changes with increased tokens per observation. Here, the CER reduces up to the maximum of 200 tokens per observation.

4.2 Comparing models trained on different datasets

The model trained on $\text{CER} = 0.1$ and $\text{WER} = 0.2$ was used as comparison against the models trained on real data. As shown in Fig. 6, the synthetic data outperforms all models trained on real data, both in terms of WER and CER. In addition, it is worth noting that the BLN600 data did not perform much better than the baseline, which could be related to the short token lengths of the sequenced text. In addition, whilst the larger SMH dataset outperformed the CA dataset, the combined CA and SMH dataset was the best performing of the three but, interestingly, only really improved on WER, not CER. It should be noted that from [11], the state-of-the-art is Claude Opus, which had a CER of 0.07 and a WER of 0.15, substantially outperforming the synthetic data. Similar to the earlier analysis, the data was split into high and low

corruption; the synthetic data performed best with the low corruption but dropped to third in the high corruption subset. In contrast, the BLN600 went from last on the low corruption subset to first on the high corruption subset, outperforming the base Llama by 8 percentage points. A figure showing the high-low split is shown in supplementary material 6.

5 Discussion

The use of synthetic data for fine-tuning LMs to perform CLOC-C has produced several findings. First, using synthetic data does improve the performance of an LM on real OCR data; the median performance of the top 10 models produced improved the CER of the base Llama model by 55% and the WER by 32%. Second, it is not only the amount of synthetic data that is important but also the distribution of the corruption in the data. Third, it is important to have sufficient tokens in each observation.

Beyond the main finding that synthetic data can be used to fine-tune LMs to perform CLOC-C, it was interesting to observe how the data distribution played an important role in the final performance. Models with relatively low levels of corruption ($\text{CER} < 0.2$) and WER in the low to mid-range ($0.2 < \text{WER} < 0.6$) seemed to perform well, with the best models reducing CER by 30% and WER by 60%. Part of this reason may be the different drivers of text correction for texts with low or high levels of corruption. For highly corrupted text, simply accurately identifying any words improves the WER. Still, substantial reductions may not be possible due to the lack of information in the text. In contrast, at low levels of corruption, the CER is improved by the LM acting as what is effectively a contextual spell check; however, as the error

CER by tokens per observation and total tokens in dataset

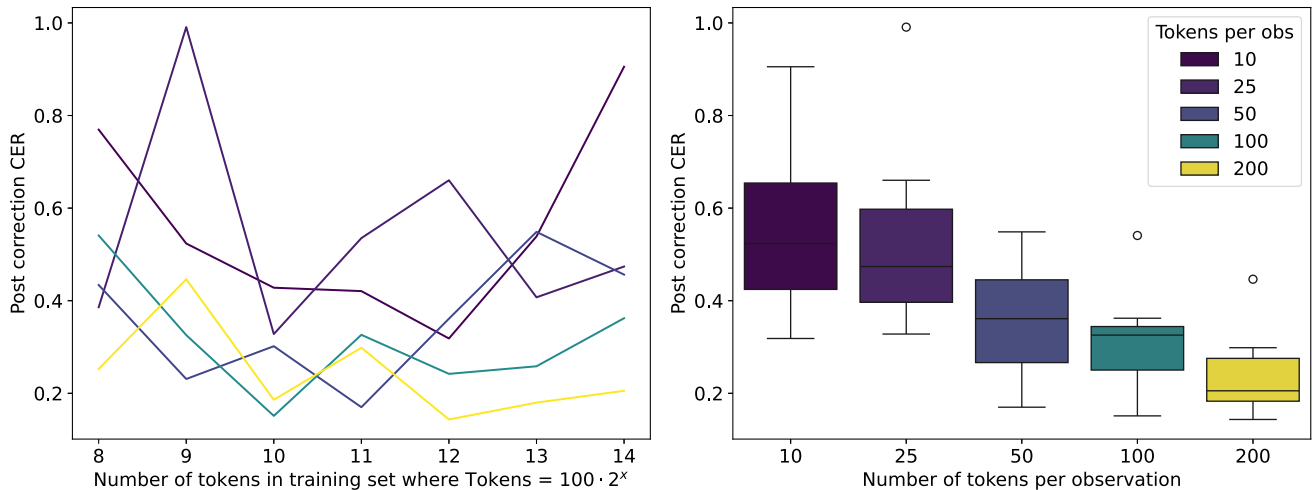


Fig. 5 The figures shown here illustrate the interaction between the number of tokens per observation and the total number of tokens in the whole training set. Increasing the number of tokens in the dataset

improves model performance. However, the impact of the total number of tokens can be difficult to assess given the overall level of noise

rate is so low, a single incorrect word can have a large relative impact on overall performance.

When training LMs, a general rule is that more diverse data leads to better-performing models. Therefore, it was surprising that the blended models had lackluster performance, with neither managing to beat the baseline median CER of 0.17. This unexpected result suggests that a better approach than training a single model on a mixture of data might be to train a mixture-of-experts model [51, 52]. Such an approach could work better as it would be able to explicitly route tasks to sub-models trained on data in the appropriate corruption range.

The exploration of the length of training context was revealing and supported the findings of [45] and [11], as models trained on observations with fewer tokens performed worse than those trained with more tokens, even though the total token training budget was fixed.

In this paper, the models were trained on text strings of 200 tokens; this is substantially less than most of the articles in the test set; it may be that, at inference time, some of the longer texts were corrected poorly due to the length itself and not the error distribution. However, such an investigation is beyond the scope of this paper. Another limitation of the paper is that the corruption function does not transpose entire words; this is a common feature of OCR errors. However, *scrambledtext* could be combined with approaches such as WildNLP [28] in a pipeline to include transposition.

Comparing the models showed how valuable synthetic data is, with the synthetic model outperforming all the real datasets. Whilst it can be said that synthetic data had an advantage in training on more tokens than the other models,

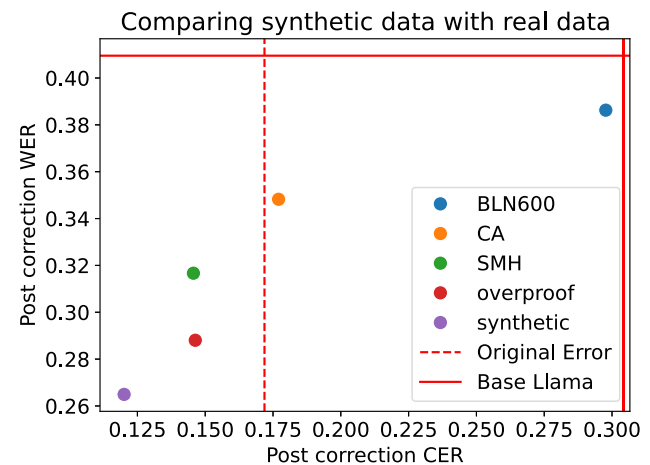


Fig. 6 When comparing models trained on synthetic data against the models trained on several existing datasets, the advantages of the synthetic data are clear with a lower CER and WER. Solid red lines show the performance of the baseline Llama3 model, and the dashed red line shows the median CER of the NCSE dataset

this is one of the key advantages of synthetic data. The comparison also highlighted the challenge faced in creating real datasets, especially when using automatic sequencing, which is a common approach [5, 9, 53]. One interesting advantage that SMH and CA datasets was being line aligned; this allowed the data to be broken into almost arbitrary lengths of coherent text, something that is not possible with either [11] or [22]. The fact that *scrambledtext* can flexibly corrupt data independent of the amount of corruption in the text it was trained on shows the value of using a Markov model,

which can be easily re-normalised to tune the corruption to the desired amount.

Although the synthetic data performed very well, it is still a way off beating current SOTA Claude Opus, which scored a CER of 0.07 and WER of 0.15, compared to the synth dataset's CER = 0.12 and WER 0.26 on the same dataset. A significant portion of the gap between Opus and the synthetically trained Llama model is the number of parameters, with Opus likely two orders of magnitude larger. Training and architecture could play a role, but as Opus is not an open-source model, exploring these differences is likely impossible.

One element of the process that could be improved to close the gap with the state-of-the-art is the corruption function itself. The method described in Sect. 3.2.1 is very simple and may explain the relatively poor performance compared to the other datasets in the high corruption subset. Perhaps a more sophisticated or nuanced approach may produce more realistic errors, improving model performance across the corruption range. Such work could also explore the ability to preserve historical spelling variations which is touched on in Supplementary material section 5.

Although this paper shed light on how to train LMs to perform CLOCR-C it did not provide any insight into the mechanisms by which the LM's perform the task. The question is therefore still open as to whether the LM is essentially acting as a stochastic parrot [54] or whether it is using a more fundamental understanding of language. This is an interesting question given the recent research by [55] into how LMs respond to impossible languages. It may be that one of the mechanisms used in CLOCR-C is that through fine-tuning, the LM gains some understanding of the structure of the "impossible language" that is the OCR errors and learns to relate this to its already existing language representation and, in doing reveal the latent "real language" masked by the errors. Further work could probe this aspect as advances could provide insight into LM language understanding and optimising for CLOCR-C.

Finally although this paper did not seek to optimise the performance, the findings open the door for work exploring optimisation and comparing model architecture. This is made easier because, as is shown in Fig. 5, the model trained on observations of 200 tokens still beats the baseline when trained on only 512 examples, roughly 5% of the full dataset dramatically reducing training cost. Such work could then more concretely compare CLOCR-C with synthetic data against the state-of-the-art.

6 Recommendations for training CLOCR-C models

Given the results of the paper, below are some heuristics to consider when using synthetic data to train CLOCR-C models.

1. **Know your data:** Model performance is dependent on the corruption level of the data were trained on and are being applied to. Having an idea of the overall level of corruption in the data can help tune the synthetic data to maximise model performance.
2. **Corruption level:** Train models on relatively low levels of corruption, a global CER of between 5–20% should produce good results on most texts.
3. **Not enough corruption is better than too much:** When creating synthetic data, training on overly corrupted data produces worse results than somewhat under-corrupted data.
4. **Corruption distribution:** Concentrating corruption into fewer words generally gives better results than spreading the corruption more evenly; this may be because the effect is more similar to masked language modelling and so is more challenging. A CER-WER pair that gives an effective CER between 0.2 and 0.6 seems to perform well.
5. **Synthetic text-length:** For a fixed token budget, having fewer observations containing more tokens provides better model performance than having more observations comprising fewer tokens.
6. **Amount of observations:** Modern LMs are very well trained and don't need many examples; training on 1000 or fewer examples (low 100k tokens, not millions) is reasonable, possibly less than 500; however, with so few examples, the difference in training time will be a few minutes so better to err on the side of caution and, at least initially stick to a slightly higher number before working your way down if necessary.
7. **Drop heavily corrupted texts:** Depending on the distribution of your data, it may be worth splitting the dataset into lightly and heavily corrupted datasets or dropping the heavily corrupted texts entirely. This is because models appear to specialise in one of the two types, and lightly corrupted models have a significantly larger relative increase in quality compared to heavily corrupted texts, which may be corrected to something semantically different to the original, causing issues in downstream analysis.
8. **Line align texts at transcription:** Whilst not strictly part of synthetic data, it is still a valuable consideration. Line aligning means that text can be much more easily broken into sub-sequences without relying on automatic sequencing algorithms. Line-aligned text can also be used to split test-sets, creating more observations allowing better diag-

nostics on why models perform better or worse in various scenarios.

7 Conclusions

This paper has shown that Language Models can be fine-tuned to perform Context Leveraging OCR Correction on entirely synthetic data. That is, training data from texts generated using a language model and then corrupted using a learned Markov corruption function. This has the potential to substantially reduce the time and cost necessary to perform an OCR correction project on a digital archive and supports the findings of previous work on training LMs with synthetic data [31–33] but applied to the OCR correction domain. The experiments developed here resulted in 68 fine-tuned models and provided a comprehensive exploration of the parameter space, which gave sufficient insight into the behaviour of the LM trained using synthetic data to allow for the creation of 8 heuristics to guide practitioners when training a CLOCR-C model. The symbol agnostic, `scrambledtext` package, makes it easy to generate synthetically corrupted training texts in any language. This paper indicates that the cost of fine-tuning a model of 8 billion parameters is low, in the range of 10 cents to a few dollars, and the bulk of the cost of a recovery project will come from inference. As such, those seeking to recover corrupted archives should focus on efficient inference strategies. These strategies may involve taking advantage of advancements in model architecture [56–59] or hardware [60–63]. By demonstrating the effectiveness of synthetic data in training CLOCR-C models, reduces the time and cost for projects repairing already digitised archives, making these valuable historical resources more accessible to researchers and the public alike.

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10032-025-00522-0>.

Acknowledgements I would like to thank James Hale for his valuable feedback on this manuscript.

Author Contributions Responsible for all aspects of the paper.

Data Availability The `scrambledtext` library is available from <https://github.com/JonnoB/scrambledtext>, The synthetic articles and other data are available from the UCL data repository [41], the github repo of the main code is available from https://github.com/JonnoB/scrambledtext_analysis, a repo of code used for training the models is available from https://github.com/JonnoB/training_lms_with_synthetic_data and can be launched directly as a lightning.ai studio.

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as

long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Smith, DA., Cordell, R.: A Research Agenda for Historical and Multilingual Optical Character Recognition. North Eastern University, <http://hdl.handle.net/2047/D20297452> (2018)
2. Chiron, G., Doucet, A., Coustaty, M., et al.: Impact of OCR Errors on the Use of Digital Libraries: Towards a Better Access to Information. In: 2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL), pp 1–4, <https://doi.org/10.1109/JCDL.2017.7991582>, <https://ieeexplore.ieee.org/document/7991582> (2017)
3. Traub, M.C., van Ossenbruggen, J., Hardman, L.: Impact analysis of OCR quality on research tasks in digital archives. In: Kapidakis, S., Mazurek, C., Werla, M. (eds.) Research and advanced technology for digital libraries. Springer International Publishing, Cham (2015)
4. Hill, M.J., Hengchen, S.: Quantifying the impact of dirty OCR on historical text analysis: Eighteenth century collections online as a case study. *Digit. Scholarsh. Humanit.* **34**(4), 825–843 (2019)
5. Chiron, G., Doucet, A., Coustaty, M., et al.: ICDAR2017 Competition on Post-OCR Text Correction. In: 2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), pp 1423–1428, <https://doi.org/10.1109/ICDAR.2017.232>, <https://ieeexplore.ieee.org/document/8270163>, ISSN: 2379-2140 (2017)
6. Nguyen, T.T.H., Jatowt, A., Coustaty, M., et al.: Survey of post-OCR processing approaches. *ACM Comput. Surv.* **54**(6), 124–137 (2021)
7. Neudecker, C., Baierer, K., Gerber, M., et al.: A survey of OCR evaluation tools and metrics. In: Proceedings of the 6th International Workshop on Historical Document Imaging and Processing. Association for Computing Machinery, New York, NY, USA, HIP '21, pp 13–18, <https://doi.org/10.1145/3476887.3476888>, <https://dl.acm.org/doi/10.1145/3476887.3476888> (2021)
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention is all you need. *arXiv: 1706.03762* [cs] (2017)
9. Thomas, A., Gaizauskas, R., Lu, H.: Leveraging LLMs for post-OCR correction of historical newspapers. In: Proceedings of the Third Workshop on Language Technologies for Historical and Ancient Languages (LT4HALA) @ LREC-COLING-2024. ELRA and ICCL, Torino, Italia, pp 116–121 (2024)
10. Boros, E., Ehrmann, M., Romanello, M., et al.: Post-correction of Historical Text Transcripts with Large Language Models: An Exploratory Study. *LaTeX-CLIL 2024* ISBN: 9798891760691 Meeting Name: The 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature Publisher: The Association for Computational Linguistics (2024)
11. Bourne, J.: CLOCR-C: Context Leveraging OCR Correction with Pre-trained Language Models. (2024), *arXiv: 2408.17428v1*
12. Dereza, O., Ní Chonghaile, D., Wolf, N.: “To Have the ‘Million’ Readers Yet”: Building a Digitally Enhanced Edition of the Bilingual Irish-English Newspaper an Gaodhal (1881–1898). In: Sprugnoli R, Passarotti M (eds) Proceedings of the Third Workshop

- on Language Technologies for Historical and Ancient Languages (LT4HALA) @ LREC-COLING-2024. ELRA and ICCL, Torino, Italia, pp 65–78, <https://aclanthology.org/2024.lt4hala-1.9> (2024)
13. Zhao, J., Wang, T., Abid, W., et al.: LoRA Land: 310 Fine-tuned LLMs that Rival GPT-4, A Technical Report. [arXiv:2405.00732](https://arxiv.org/abs/2405.00732) [cs] (2024)
 14. Fu, Z., Yang, H., So, A.M.C., et al.: On the Effectiveness of Parameter-Efficient Fine-Tuning. *Proceedings of the AAAI Conference on Artificial Intelligence* **37**(11), 12799–12807 (2023). <https://doi.org/10.1609/aaai.v37i11.26505>, <https://ojs.aaai.org/index.php/AAAI/article/view/26505>, number: 11
 15. Han, Z., Gao, C., Liu, J., et al.: Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. <https://doi.org/10.48550/arXiv.2403.14608>, [arXiv:2403.14608](https://arxiv.org/abs/2403.14608) [cs] (2024)
 16. Dettmers, T., Lewis, M., Belkada, Y., et al.: LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. [arXiv: 2208.07339](https://arxiv.org/abs/2208.07339), [cs] (2022)
 17. Biderman, D., Ortiz JG, Portes, J., et al.: LoRA Learns Less and Forgets Less. [arXiv:2405.09673](https://arxiv.org/abs/2405.09673) [cs] (2024)
 18. Zheng, Y., Zhang, R., Zhang, J., et al.: LlamaFactory: unified efficient fine-tuning of 100+ language models. [arXiv:2403.13372](https://arxiv.org/abs/2403.13372) [cs] (2024)
 19. Evershed, J., Fitch, K.: Correcting noisy OCR: context beats confusion. In: *Proceedings of the First International Conference on Digital Access to Textual Cultural Heritage*. Association for Computing Machinery, New York, NY, USA, DATECH '14, pp 45–51, <https://doi.org/10.1145/2595188.2595200>, <https://dl.acm.org/doi/10.1145/2595188.2595200> (2014)
 20. Rigaud, C., Doucet, A., Coustaty, M., et al.: ICDAR 2019 Competition on Post-OCR Text Correction. In: *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pp 1588–1593, <https://doi.org/10.1109/ICDAR.2019.00255>, <https://ieeexplore.ieee.org/document/8978127>, ISSN: 2379-2140 (2019)
 21. Jiang, M., Dubnick, R., Worthey, G., et al.: A prototype Gutenberg-Hathitrust sentence-level parallel corpus. Tech. rep., University of Illinois, <https://databank.illinois.edu/datasets/IDB-1685085>, publisher: University of Illinois at Urbana-Champaign (2021)
 22. Booth CW, Thomas, A., Gaizauskas, R.: BLN600: A Parallel Corpus of Machine/Human Transcribed Nineteenth Century Newspaper Texts. In: Calzolari N, Kan MY, Hoste V, et al.: (eds) *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*. ELRA and ICCL, Torino, Italia, pp 2440–2446, <https://aclanthology.org/2024.lrec-main.219> (2024)
 23. Bagla, K., Gupta, S., Kumar, A., et al.: Noisy Text Data: foible of popular Transformer based NLP models. In: *Proceedings of the Third International Conference on AI-ML Systems*. Association for Computing Machinery, New York, NY, USA, AIMLSys '23, pp 1–5, <https://doi.org/10.1145/3639856.3639889>, (2024)
 24. Li, X., Liu, M., Gao, S.: Can pretrained language models derive correct semantics from corrupt subwords under Noise? [arXiv:2306.15268](https://arxiv.org/abs/2306.15268) [cs] (2023)
 25. Li, J., Ji, S., Du, T., et al.: TextBugger: generating adversarial text against real-world applications. In: *Proceedings 2019 Network and Distributed System Security Symposium*, <https://doi.org/10.14722/ndss.2019.23138>, [arXiv:1812.05271](https://arxiv.org/abs/1812.05271) [cs] (2019)
 26. Huang, H., Liang, T., Sun, X., et al.: NoisyAG-News: A benchmark for addressing instance-dependent noise in text classification. [arXiv:2407.06579](https://arxiv.org/abs/2407.06579) [cs] (2024)
 27. Náplava, J., Popel, M., Straka, M., et al.: Understanding model robustness to user-generated noisy texts. [arXiv:2110.07428](https://arxiv.org/abs/2110.07428) [cs] (2021)
 28. Rychalska, B., Basaj, D., Gosiewska, A., et al.: Models in the wild: on corruption robustness of neural NLP systems. In: Gedeon T, Wong KW, Lee M (eds) *Neural Information Processing*. Springer International Publishing, Cham, pp 235–247, https://doi.org/10.1007/978-3-030-36718-3_20 (2019)
 29. Edunov, S., Ott, M., Auli, M., et al.: Understanding Back-Translation at Scale. In: Riloff E, Chiang D, Hockenmaier J, et al.: (eds) *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, pp 489–500, <https://doi.org/10.18653/v1/D18-1045>, <https://aclanthology.org/D18-1045> (2018)
 30. Nguyen, TTH., Jatowt, A., Coustaty, M., et al.: Deep statistical analysis of OCR errors for effective post-OCR processing. In: *2019 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*, pp 29–38, <https://doi.org/10.1109/JCDL.2019.00015>, <https://ieeexplore.ieee.org/abstract/document/8791206> (2019)
 31. Gunasekar, S., Zhang, Y., Aneja, J., et al.: Textbooks are all you need. <https://doi.org/10.48550/arXiv.2306.11644>, [arXiv:2306.11644](https://arxiv.org/abs/2306.11644) [cs] (2023)
 32. Li, Y., Bubeck, S., Eldan, R., et al.: Textbooks are all you need II: phi-1.5 technical report. <https://doi.org/10.48550/arXiv.2309.05463>, [arXiv:2309.05463](https://arxiv.org/abs/2309.05463) [cs] (2023)
 33. Tan, C., Wang, J.: 1.5-Pints technical report: pretraining in days, not months – your language model thrives on quality data. <https://doi.org/10.48550/arXiv.2408.03506>, [arXiv:2408.03506](https://arxiv.org/abs/2408.03506) [cs] (2024)
 34. Shumailov, I., Shumaylov, Z., Zhao, Y., et al.: AI models collapse when trained on recursively generated data. *Nature* **631**(8022), 755–759 (2024)
 35. Hart MS, Team tPG.: Project gutenberg. <https://www.gutenberg.org> (1971)
 36. Evershed, J., Fitch, K.: OverProof - Evaluation. <http://overproof.projectcomputing.com/evaluation> (2014)
 37. Holley, R.: Many hands make light work : public collaborative OCR text correction in Australian Historic Newspapers. National Library of Australia, <http://eprints.nla.gov.au/eprint.nla?id=12907> (2009)
 38. Ott, M., Edunov, S., Baevski, A., et al.: fairseq: A fast, extensible toolkit for sequence modeling. In: Ammar W, Louis A, Mostafazadeh N (eds) *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (Demonstrations)*. Association for Computational Linguistics, Minneapolis, Minnesota, pp 48–53, <https://doi.org/10.18653/v1/N19-4009>, <https://aclanthology.org/N19-4009> (2019)
 39. Gupte, A., Romanov, A., Mantravadi, S., et al.: Lights, camera, action! a framework to improve nlp accuracy over ocr documents. <https://doi.org/10.48550/arXiv.2108.02899>, [arXiv:2108.02899](https://arxiv.org/abs/2108.02899) [cs] (2021)
 40. Lovelace, A.: Sketch of the analytical engine invented by Charles Babbage, Esq./notes by the translator. https://en.wikisource.org/wiki/Scientific_Memoirs/3/Sketch_of_the_Analytical_Engine_invented_by_Charles_Babbage,_Esq./Notes_by_the_Translator (1842)
 41. Bourne, J.: Scrambled text Datasets from the paper. (2024). <https://doi.org/10.5522/04/27108334.v1>
 42. Strickland, E.: Andrew Ng: Unbiggen AI - IEEE Spectrum. <https://spectrum.ieee.org/andrew-ng-data-centric-ai> (2022)
 43. Zha, D., Bhat, ZP., Lai, KH., et al.: Data-centric Artificial Intelligence: A Survey. <https://doi.org/10.48550/arXiv.2303.10158>, [arXiv:2303.10158](https://arxiv.org/abs/2303.10158) [cs] (2023)
 44. Salehi, S., Schmeink, A.: Data-centric green artificial intelligence: a survey. *IEEE Transactions on Artificial Intelligence* **5**(5), 1973–1989 (2024). <https://doi.org/10.1109/TAI.2023.3315272>, <https://ieeexplore.ieee.org/abstract/document/10251541>, conference Name: IEEE Transactions on Artificial Intelligence
 45. Dubey, A., Jauhri, A., Pandey, A., et al.: The Llama 3 Herd of Models. <https://doi.org/10.48550/arXiv.2407.21783>, [arXiv:2407.21783](https://arxiv.org/abs/2407.21783) [cs](2024)

46. Hu, E.J., Shen, Y., Wallis, P., et al.: LoRA: low-rank adaptation of large language models. <https://doi.org/10.48550/arXiv.2106.09685>, arXiv:2106.09685 [cs] (2021)
47. Kalajdziewski, D.: A rank stabilization scaling factor for fine-tuning with LoRA. <https://doi.org/10.48550/arXiv.2312.03732>, arXiv:2312.03732 [cs] (2023)
48. Han, D., Han, M.: Unsloth. <https://docs.unsloth.ai/get-started/all-our-models> (2024)
49. Simpson, E.H.: The interpretation of interaction in contingency tables. *J. R. Stat. Soc.: Series B (Methodol.)* **13**(2), 238–241 (1951). <https://doi.org/10.1111/j.2517-6161.1951.tb00088.x>
50. Blyth, C.R.: On Simpson's paradox and the sure-thing principle. *J. Am. Stat. Assoc.* **67**(338), 364–366 (1972)
51. Jacobs, R.A., Jordan, M.I., Nowlan, S.J., et al.: Adaptive mixtures of local experts. *neural computation* **3**(1), 79–87 (1991). <https://doi.org/10.1162/neco.1991.3.1.79>, <https://ieeexplore.ieee.org/abstract/document/6797059>, conference Name: Neural Computation
52. Jiang AQ, Sablayrolles, A., Roux, A., et al.: Mixtral of experts. <https://doi.org/10.48550/arXiv.2401.04088>, arXiv:2401.04088 [cs] (2024)
53. Jiang, M., Hu, Y., Worthey, G., et al.: The Gutenberg-HathiTrust parallel corpus: a real-world dataset for noise investigation in uncorrected OCR texts. <https://hdl.handle.net/2142/109695>, publisher: iSchools (2021)
54. Bender EM, Gebru, T., McMillan-Major, A., et al.: On the Dangers of Stochastic Parrots: Can Language Models Be Too Big? In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. Association for Computing Machinery, New York, NY, USA, FAccT '21, pp 610–623, <https://doi.org/10.1145/3442188.3445922>, (2021)
55. Kallini, J., Papadimitriou, I., Futrell, R., et al.: Mission: impossible language models. <https://doi.org/10.48550/arXiv.2401.06416>, arXiv:2401.06416 [cs] (2024)
56. Shah, J., Bikshandi, G., Zhang, Y., et al.: FlashAttention-3: fast and accurate attention with asynchrony and low-precision. <https://doi.org/10.48550/arXiv.2407.08608>, arXiv:2407.08608 [cs] (2024)
57. Gu, A., Dao, T.: Mamba: linear-time sequence modeling with selective state spaces. <https://doi.org/10.48550/arXiv.2312.00752>, arXiv:2312.00752 [cs] (2024)
58. Dao, T., Gu, A.: Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. arXiv: 2405.21060v1 (2024)
59. Ma, S., Wang, H., Ma, L., et al.: The Era of 1-bit LLMs: all large language models are in 1.58 Bits. <https://doi.org/10.48550/arXiv.2402.17764>, arXiv:2402.17764 [cs] (2024)
60. SambaNova: accelerated computing with a reconfigurable dataflow architecture. Tech. rep., SambaNova, https://sambanova.ai/hubfs/23945802/SambaNova_Accelerated-Computing-with-a-Reconfigurable-Dataflow-Architecture_Whitepaper_English-1.pdf (2021)
61. Hall, S., Schreiber, R., Lie, S., et al.: Training Giant Neural Networks Using Weight Streaming on Cerebras Wafer-Scale Clusters. Tech. rep., Cerebras, <https://8968533.fs1.hubspotusercontent-na1.net/hubfs/8968533/Virtual%20Booth%20Docs/CS%20Weight%20Streaming%20White%20Paper.pdf> (2024)
62. Yang, Z., Mellachervu, K., Arsovski, I., et al.: Optimized Simulation Methodology of Warpage and Localized Stress Hotspot Prediction for Assembly Risk Assessment. In: 2024 IEEE 74th Electronic Components and Technology Conference (ECTC). IEEE, Denver, CO, USA, pp 1011–1017, <https://doi.org/10.1109/ectc51529.2024.00162>, <https://ieeexplore.ieee.org/document/10564995/> (2024)
63. NVIDIA: NVIDIA blackwell architecture technical overview. Tech. rep., NVIDIA, <https://resources.nvidia.com/en-us-blackwell-architecture> (2024)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.